CS3604 12306 项目开发报告

成员：陈卓妍 523030910
李青雅 523030910004
侯湘贻 523030910

2026 年 1 月 15 日

目录

1 项目简介 **3**

2 需求管理 **3**

2.1 从代码和 12306 页面反推需求 3

2.2 用 YAML 文件代替传统文档 3

2.3 进度追踪 4

3 需求规模 **5**

3.1 顶层模块数量 5

3.2 子需求和场景数量 5

4 智能体辅助设计与测试 **6**

4.1 MCP 工具链的使用 6

4.2 AI 怎么帮我们干活 6

4.3 如何保证可复现 6

5 测试用例与通过率 **7**

5.1 测试情况 7

5.2 测试结果 7

6 总结 **8**

1 项目简介

本项目基于课程提供的 12306 基础代码，在有限时间内完成了一个可运行、可复现的购票系统原型，涵盖了“查票—下单—支付—餐饮”的主要使用场景。

我们的主要工作不仅仅是“写代码”，更重要的是尝试了一套新的开发模式：

1. **用代码管理需求**：把需求写成 YAML 文件，而不是 Word 文档。
2. **让智能体辅助开发**：利用 MCP 工具链，让 AI 能够理解我们的需求队列，并辅助我们做模块设计和测试撰写。
3. **保证可复现性**：通过固定的数据库快照（sqlite），确保任何人（包括老师和助教）拿到代码后，都能看到一样的车次和数据，复现一样的功能。

本报告围绕老师提出的四个问题展开：

- 我们是**如何进行需求管理**的；
- 当前系统的**需求规模**大致到什么程度；
- 我们是如何**用智能体来完成模块设计和测试撰写**的；
- 目前**测试用例和通过情况**如何。

2 需求管理

2.1 从代码和 12306 页面反推需求

一开始我们没有从零写一份很厚的 SRS，而是先做了两件事：

- 通读课程给的前后端代码，看现有实现到底支持哪些功能。
- 对照真正的 12306 网站，梳理出“首页—登录—查票—下单—支付—个人中心—忘记密码—餐饮”的主流程。

在这个基础上，我们把“已经实现的功能”和“仍然想做的功能”整理成一棵需求树，再用代码的形式落在 YAML 里。

2.2 用 YAML 文件代替传统文档

传统的软件工程作业中，需求文档（SRS）往往写完就丢在一边，和代码严重脱节。为了解决这个问题，我们建立了一棵“活着的需求树”，存放在 `requirements_mcp.yaml` 文件中。

这个文件主要包含：

- **需求节点**：每个功能点都有唯一的 ID（如 `REQ-UI-3`）。
- **场景描述**：每个功能点下都有具体的 Scenario，描述用户怎么操作（Action）以及系统应该有什么反应（Expectation）。
- **代码映射**：我们在需求里直接写明了对应的 UI 文件路径（如 `TrainListPage.tsx`）和后端接口。

这样做的最大好处是：AI 可以直接读取这个文件，知道当前项目有哪些功能，每个功能对应哪些代码文件。

具体例子包括：

- ROOT 和 REQ-ENV-1 约定了必须使用根目录的 `database.sqlite`，并检查里面是否包含 G101、D313 等示例车次。
- REQ-UI-3（车票查询）要求从首页携带 `from/to/date` 参数跳转到 `/train-list`，并通过 `TrainListPage.tsx` 和 `trainService.searchTrains` 渲染列表。
- REQ-UI-7（餐饮特产）把 `/catering`、`/catering/book`、`/catering/vendor` 三类页面都写在描述里，方便从需求直接跳到对应的 React 组件。

2.3 进度追踪

为了搞清楚项目做到了哪一步，我们在 `artifacts` 目录下维护了两个简单的辅助文件：

- `architecture_graph.yaml`：描述需求之间的父子关系，可以生成树状图。
- `phase_one_progress.yaml`：记录每个需求是“已处理”还是“未处理”，相当于一个任务进度表。

整体来看，我们的需求管理过程可以概括为四步：

1. 从已有代码和真实 12306 页面反推主流程；
2. 把这些流程整理成一棵带 ID 的需求树，并用 YAML 固化下来；
3. 在需求里嵌入具体的 UI 文件和接口路径，让需求、设计和实现对应得上；
4. 用 `artifacts` 目录下的图结构和进度文件，记录每个需求“做到什么程度了”，方便后续用智能体继续接力。

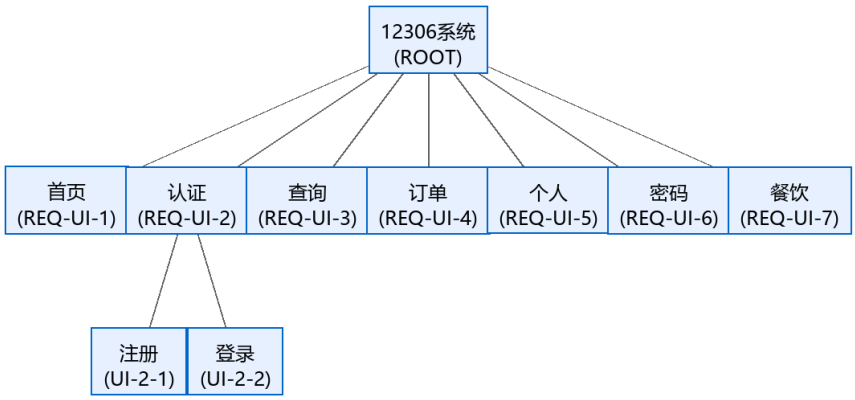


图 1: 需求结构图（示意）

3 需求规模

从需求树结构来看，我们在项目中覆盖了 12306 的核心业务流程，主要包括以下几层：

3.1 顶层模块数量

顶层的功能模块（ROOT 的子节点）包括：

1. **REQ-UI-1 首页**：系统入口与导航。
2. **REQ-UI-2 用户认证**：包含登录、注册页面。
3. **REQ-UI-3 车票查询**：核心功能，支持按城市、日期查询车次，并进行筛选。
4. **REQ-UI-4 订单流程**：从预订、填写乘车人到提交订单。
5. **REQ-UI-5 个人中心**：管理个人信息和乘车人列表。
6. **REQ-UI-6 忘记密码**：模拟了找回密码的四步流程。
7. **REQ-UI-7 餐饮特产**：这是我们额外扩展的模块，包含餐饮首页、商家列表、商品选择等功能。

此外还有 REQ-ENV-1（环境与数据库初始化），用来单独固定运行时所需的数据状态。

3.2 子需求和场景数量

在顶层之下，我们又拆出若干子需求和具体场景，例如：

- 用户认证下拆出了 REQ-UI-2-1（注册）和 REQ-UI-2-2（登录），分别对应独立的页面和接口。
- 忘记密码模块用 REQ-UI-6:SCE-0~3 四个场景覆盖了“填写账号信息—输入验证码—设置新密码—完成提示”四步。
- 车票查询模块（REQ-UI-3）中用 SCE-0~2 分别描述了页面初始化、筛选条件变化和点击“预订”跳转订单页。

整个 YAML 文件里，每个需求节点下至少有 1 个场景，完整覆盖了用户从打开首页到完成购票、管理乘车人、下单餐饮的主要路径。相比只写“功能列表”，这样的拆分更贴近真实用户操作，也更方便后续写测试。

总体来看，我们没有刻意堆砌“需求点数量”，而是围绕一条完整业务链路，把关键节点拆细，并保证每个节点都有对应的 UI 页面和接口实现。

如果从“规模”的角度简单总结，可以认为：

- 顶层功能模块：1 个 ROOT + 1 个环境需求 + 7 个核心 UI 模块；
- 子需求：在认证、忘记密码等模块下拆出了若干更细的子节点（如注册、登录、四步向导等）；
- 场景数量：每个节点至少 1 个场景，覆盖了从打开首页到完成购票、管理乘车人和下单餐饮的一整条用户路径。

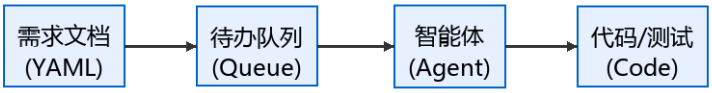


图 2: 智能体辅助开发流程

4 智能体辅助设计与测试

4.1 MCP 工具链的使用

为了让 AI 更好地帮助我们，我们引入了 MCP（Model Context Protocol）工具链。简单来说，就是通过几个脚本让 AI “听懂”我们的指令，以需求为入口去看代码：

- `init_top_down_queue`：读取需求 YAML，生成一个待办任务队列。
- `pop_next_requirement`：告诉 AI “下一个要做什么需求”。
- `register_ui_component`：告诉 AI “这个页面的 UI 组件在哪里，参数是什么”。

基于这套工具链，我们可以让 AI 按顺序“从 ROOT 走到 REQ-UI-1~7”，每次只关注一个需求节点，减少遗漏。

4.2 AI 怎么帮我们干活

在实际开发中，AI 主要帮我们做了两件事：

1. **模块设计**：比如在做餐饮模块时，我们把需求告诉 AI，它会根据现有的代码风格，帮我们规划出需要新建哪些 Page 和 Component，甚至帮我们写出路由配置。
2. **测试辅助**：AI 知道每个需求的“期望结果”是什么，所以它可以帮我们生成测试思路。比如“在这个场景下，应该检查哪个按钮是否存在，或者哪个接口是否返回 200”。

4.3 如何保证可复现

为了让老师和助教能稳定复现我们的运行结果，我们做了一个关键的决定：**把数据库文件纳入版本控制**。

- 我们在根目录下准备了一个 `database.sqlite` 文件。
- 在需求文档的第一项（REQ-ENV-1）就明确规定：系统启动时必须使用这个数据库文件。

这样，无论谁来运行我们的代码，看到的都是同一天的车次、同一个价格，不会出现“我这里能查到票，你那里查不到”的情况。对 AI 而言，也只需要照着 REQ-ENV-1 先把数据库就绪，再依次跑各个 UI 场景，就能复现实验结果。

5 测试用例与通过率

5.1 测试情况

我们主要进行了三类测试：

- **后端接口测试**：针对 Login、Query、Order 等核心 API，验证数据返回是否正确。
- **前端自动化测试**:使用前端测试框架跑单元/组件测试（最近一次运行结果记录在 `frontend/test-results/.la` 中，状态为 `passed`）。
- **界面功能测试**：手动验证了从登录到下单的完整流程，确保页面跳转逻辑通顺。

特别是在餐饮模块，我们重点测试了：

- 能否正确根据车次筛选商家；
- 商家列表是否展示正确；
- 点击商家能否进入详情页。

5.2 测试结果

从已经编写的自动化测试来看：

- **后端覆盖率**：整体语句覆盖率约为 56.75%，其中 `src/routes` 模块的语句覆盖率接近 98%，`src/models` 模块语句覆盖率超过 90%。核心路由和模型的基本分支都被覆盖到了。
- **前端测试结果**：最近一次前端测试的整体状态为 `passed`，当前已有的测试用例全部通过。

结合自动化与手工测试，可以认为在“正常使用路径”上的测试通过率较高，核心购票流程能够稳定跑通。

- **通过**：正常的用户购票流程、乘车人添加、餐饮浏览等主路径在当前测试集下均为通过状态。
- **待改进**：部分异常输入（如非法身份证号、极端日期）的处理还不够完善；部分脚本和工具类代码的覆盖率有提升空间；GUI 自动化测试覆盖的异常场景仍然偏少。

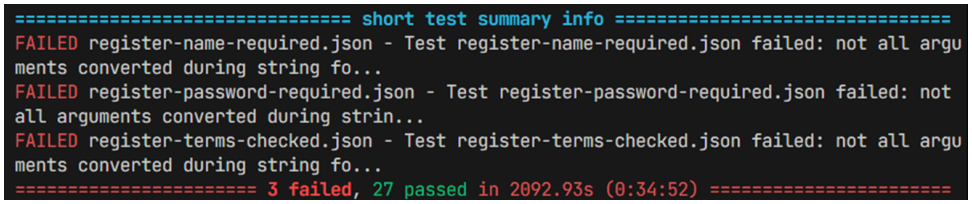


图 3: 测试执行结果截图

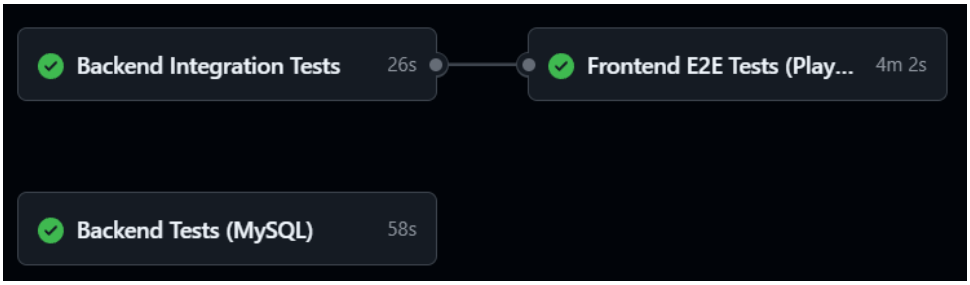


图 4: GitHub Actions CI 全部通过截图

6 总结

通过这个项目，我们不仅复刻了 12306 的核心功能，更重要的是实践了一种“文档驱动开发”的新模式。

- 需求不再是死文档，而是 AI 可读的 YAML。
- 数据库快照保证了环境的一致性。
- 智能体工具链让开发和测试变得更有条理。

这种方法让我们的项目不仅“能跑”，而且“好懂”、“好复现”。